

chapter7

제7장 배치 분석과 데이터 가공

6장 마지막에서 스트리밍의 수치가 “watermark 안에서만 지연을 수용한 근사 확정치”라는 한계를 확인했다 — 폐기된 지연 이벤트(6장의 LATE_DROP)는 대시보드 어디에도 반영되지 않는다. 그런데 공공 업무의 다른 절반은 정확히 그 반대를 요구한다: 월간 보고서, 정산, 감사 자료는 “빠른 근사치”가 아니라 “마감 후 전체를 다시 세어 확정된 수치”여야 한다. 이 장은 그 확정의 세계, 즉 축적된 데이터를 정기적으로 다시 가공하는 배치 분석을 다룬다. 먼저 배치가 스트리밍과 어떻게 역할을 나누는지 정리하고, 배치의 첫 단계인 데이터 정제와 표준 코드 매핑을 실측으로 관찰한다. 이어서 배치 절차를 태스크(task)의 그래프로 선언하는 Airflow DAG를 작성해 의존성·실패 전파·재실행을 실제 엔진에서 확인하고, 일별·주별 보고서 자동화 파이프라인의 입력과 출력을 계약으로 정의한다. 실습(★★★☆☆)은 브로커(broker)·Java 없이 단독 실행이 가능하며, Airflow가 Windows를 네이티브 지원하지 않으므로 Windows에서는 WSL2 또는 컨테이너(container)가 필요하다.

학습 목표

스트리밍을 보완하는 배치 워크플로의 역할(확정 통계·보고 자동화)을 설명하고 설계할 수 있다.
Airflow DAG의 태스크·의존성·logical date를 이해하고, 기본 DAG를 작성해 실행할 수 있다.
공공 보고서 자동화 파이프라인의 입력·출력 계약을 정의하고, 재실행 안전성(멥등·재현성)을 검증할 수 있다.

7.1 배치 처리가 필요한 공공 업무

학습 목표

스트리밍이 있는데도 배치가 필요한 이유를 확정성·재실행·전체 조망으로 설명한다.
두 방식의 역할 분담을 6장 실측과 연결해 이해한다.

직관: 상황판과 월간 보고서는 성격이 다르다

2장에서 배치를 “모아서 한 번에”, 스트리밍을 “도착하는 대로”라고 구분했고, 6장에서 스트리밍 쪽을 실제로 구현했다. 그렇다면 스트리밍이 실시간으로 모두 계수하는데 왜 같은 데이터를 배치로 또 처리하는가. 답은 두 산출물의 성격이 다르기 때문이다. 상황판의 수치는 빠르되 미확정이어도

된다 — 6장 update 모드의 창 건수는 지연 이벤트가 오면 3에서 4로 바뀌었다. 반면 월간 보고서의 수치는 늦어도 되지만 바뀌면 안 된다 — 한번 제출한 통계가 소급으로 변경되면 보고 자체를 신뢰할 수 없게 된다.

6장의 실측에서 이 차이가 확인된다. watermark 10분 설정에서 21:49 발생 이벤트(LATE_DROP)는 폐기되어 스트리밍 집계에 영원히 빠져 있다 — 폐기 카운터(1)에만 흔적이 남았다. 스트리밍 세계에서 이것은 의도된 규칙이지만, “그 날의 정확한 총 건수”를 요구하는 확정 통계에서는 1건의 누락이다. 배치는 마감 후(예: 다음 날 새벽) 그날 도착분 전체를 다시 읽으므로, watermark 너머의 지연분까지 포함해 다시 계수할 수 있다. 3장에서 설계한 하이브리드(스트리밍+배치 병행)의 실체가 바로 이 역할 분담이다.

마감(cutoff)의 관점에서 보면 두 방식의 관계가 더 선명해진다. 6장의 watermark는 **데이터가 마감을 정하는** 방식이었다 — 관측된 최대 이벤트 시각에서 지연 허용을 뺀 만큼만 기다린다. 배치의 마감은 **달력이 정한다** — “7월 1일 데이터는 7월 2일 새벽에 마감하고 계산한다”. 달력 마감은 느리지만 예측 가능하고, 마감 시점까지 도착한 모든 지연분을 포함한다. 같은 문제(언제까지 기다릴 것인가)에 대한 두 개의 다른 답이며, 그래서 두 시스템은 대체재가 아니라 보완재다.

물론 달력 마감도 완전하지는 않다. 마감 이후에 도착하는 소급 정정(예: 며칠 뒤의 접수 취소·오기 정정)은 배치도 놓친다. 그래서 확정 통계에는 개정판 절차 — 해당 날짜의 재실행과 보고서 버전 표기 — 가 함께 설계되어야 하며, 그 재실행을 안전하게 만드는 조건이 이 장의 뒷부분(멥등·재현성, 7.4)이다.

핵심 개념

확정(finality): 배치는 데이터 구간의 마감을 기다렸다가 전체를 계산하므로, 산출물이 “그 시점 입력의 최종 결과”라는 지위를 확보한다. 스트리밍의 근사치를 배치가 사후 검증하는 구조는 두 시스템의 차이를 정기적으로 대조하는 감시 지표가 된다.

재실행 가능(re-runnable): 배치의 실행 단위는 “어느 날짜의 데이터”다. 실패하면 그 날짜를 재실행하면 된다 — 단, 재실행해도 결과가 이중으로 적재되지 않아야 한다. 5장의 멥등성이 배치의 전제 조건으로 되돌아온다(7.4에서 실측).

전체 조망: 배치는 그날 데이터 전부를 한 번에 보므로, 스트리밍이 하기 어려운 계산(전체 정렬, 중복 전수 검사, 원천 대사)이 자연스럽다.

지연·처리량의 절충: 배치의 지연은 실행 주기만큼 크다(일 배치면 최대 하루). 그 대가로 큰 데이터를 효율적으로 처리하고, 산출물을 확정할 수 있다.

표 7.1 스트리밍과 배치의 역할 분담(6장·7장 실습 기준)

항목	스트리밍(6장)	배치(7장)
목적	상황 감시·조기 경보	확정 통계·보고서·정산
수치의 지위	근사치(지연분 수용 중, 갱신됨)	확정치(마감 후 전체 계산)
지연 데이터	watermark 안만 수용, 밖은 폐기	마감 시점까지 도착분 전부 포함
실패 시	체크포인트에서 재개	해당 날짜 재실행(역등 전제)
산출물	갱신되는 집계 스트림	불변 보고서 파일

주기와 실행 시각은 무엇에서 정하는가

배치를 사용하기로 정했다면 다음 질문은 “얼마나 자주, 몇 시에 실행하는가”다. 두 값을 관행으로 정하면 나중에 바꾸기 어렵다 — 주기가 산출물의 이름·파티션·재실행 단위를 함께 정하기 때문이다.

주기는 **수요 쪽의 최신성 요구**와 **공급 쪽의 비용** 사이에서 선택한다. 최신성은 산출물을 사용하는 사람이 정한다. 매일 아침 회의에서 전날 통계를 사용한다면 일 배치가 필요하고, 통계 월보만 산출한다면 일 배치는 30회 실행해 1회만 사용하는 셈이다. 반대로 주 배치를 설정하고 “어제 수치”를 요구하면 요구 자체가 성립하지 않는다. 비용은 실행 1회의 자원만이 아니다. 주기를 절반으로 줄이면 실행 횟수와 함께 실패·재실행·알림도 두 배가 된다.

실행 시각은 최신성이 아니라 **원천 데이터가 언제 확정되는가**에서 역산한다. 7월 1일분을 7월 2일 00시 정각에 실행하면 자정 직전에 접수되어 아직 적재되지 않은 건을 놓친다. 원천 시스템의 야간 적재가 새벽 1시에 끝난다면 배치 시작은 그 뒤여야 하고, 산출물을 아침 8시 회의에 쓴다면 실행 시간과 한 번의 복구에 쓸 여유가 그 사이에 들어가야 한다. 즉 실행 시각은 [원천 확정 시각, 산출물 필요 시각 - 실행 시간 - 복구 여유] 구간에서 선택한다. 이 구간이 비어 있으면 시각을 조정할 것이 아니라 주거나 마감 규약을 다시 정해야 한다.

민간의 마감 배치, 공공의 확정 보고

배치가 구식 기술로 보일 수 있으나, 민간 데이터 현장의 하루가 어떻게 마감되는지를 확인하면 그렇지 않다. 커머스·광고·핀테크 회사의 데이터 조직은 대개 **마감 배치(day-close batch)**를 둔다. 자정이 지나면 그날의 거래·노출·정산 데이터를 전량 다시 읽어 “어제의 확정 숫자”를 만들고, 이 수치가 회계·정산·경영 보고의 기준이 된다. 대표적인 세 가지가 있다 — 매출·정산 마감 배치(그날 팔린 것과 환불된 것을 대서해 최종 매출을 확정), 광고 리포트 일 배치(광고주에게 어제의 노출·클릭·비용을 확정 리포트로 발송), 데이터 웨어하우스 야간 적재(nightly load — 낮 동안 여러 운영 DB에 흩어진 데이터를 새벽에 한 번에 창고로 모아 분석용 표를 다시 만듦). 셋의 공통점은 “실시간 대시보드가 이미 대략의 수치를 제시하고 있는데도, 밤에 한 번 더 전량을 확정한다”는 것이다. 실시간은 장중(場中)의 감시용이고, 확정은 정산·보고·감사용이다.

공공의 업무는 이 마감 문화를 그대로, 오히려 더 강하게 필요로 한다. 민간의 마감에 “틀리면 다음 분기 실적에 반영해 정정”으로 끝나는 경우가 많은 반면, 공공의 확정치는 감사와 국회·의회 보고, 정보공개청구의 대상이 되므로 **한번 확정한 수치를 바꾸는 일 자체가 사건**이다. 확정치(finality)

정정이 공공에서 더 까다로운 이유가 바로 이 책임 구조다 — 누가 언제 어떤 데이터로 이 수치를 만들었고, 왜 이 값이 최종인지를 설명할 수 있어야 한다. 그래서 공공의 배치는 민간보다 재현성·원천 보존·마감 규약의 명시를 더 강하게 요구한다.

표 7.1a 민간 마감 배치의 공공 적용

민간 현장 방법	무엇을 확정하나	공공 대응물	공공 특수성에 따른 변형
매출·정산 마감 배치(day close)	그날의 최종 매출·정산액 (환불 대사 후)	회계 결산, 세입·세출 마감	감사·결산 검사 대상 — 정정 시 개정 이력과 사유가 함께 남아야 함
광고 리포트 일 배치	광고주에게 보낼 어제의 확정 성과 리포트	민원 일일 마감 보고, 부서 배포용 일별 통계	배포 후 수치 변동이 불신 요인 — 마감 후 확정·불변이 원칙(7.4 재현성)
DW 야간 적재(nightly load)	분석용 확정 데이터셋 (여러 원천 통합)	통계 월보·연보 확정치, 정책 기초통계	표준 코드로 기관 간 통합(7.2) — 명칭이 아닌 코드가 결합 키

적용의 핵심은 방법이 같고 책임의 강도가 다르다는 것이다. 민간에서 “일 마감을 실행한다”는 엔지니어가 공공으로 오면 같은 파이프라인을 구성하되, 산출물에 마감 규약·원천 스냅샷·재현 증거를 함께 남기는 층이 하나 더 붙는다 — 그 층이 이 장의 뒷부분(7.4 계약·재현성)이 다루는 내용이다. 반대로 공공을 공부한 학습자가 민간으로 가면, 이미 감사 수준의 확정 규약을 익혔으므로 마감 배치의 정합 요구를 어렵지 않게 충족한다.

공공 사례

지자체 민원 상황실은 6장의 스트리밍 집계로 “최근 10분 지역별 신고”를 보고, 기획 부서는 이 장의 배치로 “어제의 확정 일별 통계”와 “주간 보고서”를 받는다. 감사 관점에서 중요한 것은 두 수치가 다를 수 있고 달라도 되는 이유가 문서화되어 있는가다 — 상황판은 폐기·미확정을 포함하지 않는 근사치이고, 보고서는 마감 후 확정치라는 성격이 명시되어 있으면, “대시보드와 보고서 수치가 다르다”는 감사 지적은 결함이 아니라 설계로 답변된다. ### 핵심 정리 - 스트리밍은 빠른 근사치, 배치는 늦은 확정치 — 공공 업무는 둘 다 필요로 하며 서로를 검증한다. - 배치의 실행 단위는 “날짜 (데이터 구간)”이고, 재실행 안전성은 멍등성에서 나온다. - 두 수치의 차이는 숨길 결함이 아니라 문서화할 설계다.

7.2 데이터 정제와 표준 코드 매핑

학습 목표

표기 변형·결측·비표준 값이 집계를 어떻게 왜곡하는지 이해한다.

표준 코드 매핑과 “버리지 않는 분리 보고” 원칙을 실측으로 확인한다.

직관: 사람에게는 같은 값이지만 groupby에는 다른 값이다

실습 입력의 7월 1일 파일에는 강남구 민원이 세 가지 표기로 들어 있다 — “강남구”, “ 강남구 ” (앞뒤 공백), “강남” (약칭). 사람은 셋을 같은 지역으로 인식하지만, 문자열로 집계하는 프로그램에게 이것은 서로 다른 세 그룹이다. 정제 없이 지역별 건수를 계수하면 강남구 8건이 세 줄로 나뉘어, 보고서의 어떤 줄도 진실이 아니게 된다. 정제(cleaning)는 이 어긋남을 규칙으로 흡수하는 단계다: 공백을 제거하고, 약칭을 표준명으로 보정하고, 표준명을 코드로 바꾼다.

마지막 단계가 왜 필요한가. 지역“명”은 기관마다, 시스템마다, 담당자마다 표기가 달라지는 자연어다. 반면 **행정표준코드**(법정동코드)는 행정표준코드관리시스템(code.go.kr)이 관리하는 유일 식별자로, 서울 강남구는 시군구 5자리 11680, 마포구는 11440, 관악구는 11620 이다. 지역명 대신 코드를 집계·결합의 키로 사용하면, 다른 기관의 데이터(인구·예산·시설)와 결합할 때도 같은 키로 연결된다. 공공 데이터에서 “이름이 아니라 코드로 붙인다”는 원칙은 표기 문제의 근본 해결이자 기관 간 상호운용의 전제다.

같은 문제를 민간도 푼다. 여러 채널에서 들어온 상품 데이터에 같은 물건이 다른 이름·다른 ID로 실려 오면, 매출 집계가 지역명 세 표기처럼 분리된다. 그래서 커머스 회사는 표준 SKU(Stock Keeping Unit, 재고 관리 단위)로 통일하고, 이 표준값을 만들고 유지하는 시스템을 마스터 데이터 관리(MDM, Master Data Management)라 부른다. 방법은 같고 표준의 출처가 다르다 — 공공은 국가가 관리하는 코드 체계를 채택하고, 민간은 자체 마스터를 구축한다. 그래서 검수 질문도 달라진다. 공공에서는 “왜 표준 코드를 쓰지 않았는가”를 묻고, 민간에서는 “마스터가 최신인가”를 묻는다.

정제의 세 갈래 출구: 매핑, 분리, 그리고 절대 버리지 않기

정제에서 가장 위험한 코드는 매핑에 실패한 레코드를 조용히 건너뛰는 코드다. 4장에서 시나리오 B의 유실이 로그 없이 조용했고, 6장에서 watermark 폐기가 결과에 흔적을 남기지 않았듯이 — 정제 단계의 drop도 별도 기록 없이는 발견되지 않는다. 보고서 총계가 원천보다 조금 작은데 아무도 모르는 상태, 이것이 감사에서 가장 곤란한 결함이다.

그래서 이 장의 정제는 모든 입력 레코드를 네 갈래 중 하나로 보내고, 어느 갈래로 분류되었는지 모두 기록한다.

매핑 성공: 표준 코드가 붙어 집계에 들어간다.

지역 미기재(missing): 지역 필드가 비어 있다 — 건수와 민원 ID를 분리 기록한다.

매핑 실패(unmapped): 표준 코드 사전에 없는 표기(예: 서울 사전에 없는 판교) — 원문 그대로 분리 기록해 후속 조치(사전 보강 또는 반려) 대상으로 남긴다.

중복 제거(dup): 같은 민원 ID의 재전송은 먼저 온 것만 남긴다(5장의 원칙 그대로).

그리고 네 갈래의 합이 입력과 일치하는지를 **총계 보존식**으로 검증한다: $\text{입력} = \text{매핑} + \text{미기재} + \text{미매핑} + \text{중복제거}$. 7월 1일 실측으로 쓰면 $20 = 18 + 1 + 1 + 0$ 이다. 이 등식이 성립하지 않으면 정제 코드 어딘가에서 레코드가 조용히 유실된 것이므로, 실습의 품질 점검 태스크는 이 경우 배치

자체를 실패시킨다(7.3의 품질 게이트).

이 등식의 각 항은 그대로 운영 지표가 된다. 매핑 ÷ 입력이 **매핑률**이고, 이 값이 낮아지면 사전에 없는 표기가 증가했다는 뜻이다 — 관할이 실제로 넓어졌거나, 접수 화면이 바뀌어 새 표기가 들어오기 시작했거나 둘 중 하나다. 민간도 같은 지표를 사용한다. 상품 카탈로그 매핑률이 낮아지면 새 상품이 마스터에 등록되지 않은 것이고, 광고 캠페인 ID 매핑률이 낮아지면 신규 캠페인이 누락된 것이다. 다만 비율만으로는 원인을 알 수 없으므로, 값이 낮아진 날의 미매핑 원문 목록을 함께 확인한다.

워크드 예제: 7월 1일 20건의 정제(실측)

실습 7.1의 7월 1일 입력 20건이 정제를 통과한 결과는 다음과 같다 — 수치는 실제 실행 산출물 (`quality_check.json`)이다.

표 7.2 정제 규칙표(7월 1일 실측)

순서	규칙	예시(입력 파일에서)	적용 실측
1	중복 접수 제거(같은 민원 ID는 첫 건만)	— (7/1에는 없음, 7/2에 1건)	0건
2	앞뒤 공백 제거	" 강남구 " → 강남구	1건
3	약칭 보정(별칭 사전)	강남 → 강남구	4건
4	표준 코드 매핑	강남구 → 11680	18건 성공
5	지역 미기재 분리	지역 필드 없음	1건
6	매핑 실패 분리(원문 보존)	판교 — 서울 사전에 없음	1건

수작업으로 검산한다. 입력 20건 중 지역 미기재 1건과 사전 밖 표기(판교) 1건을 빼면 18건이 코드를 얻는다 — 강남구(11680) 8, 마포구(11440) 5, 관악구(11620) 5. 8+5+5=18로 매핑 건수와 일치하고, 18+1+1+0=20으로 총계 보존식도 성립한다. 사흘 전체로는 매핑 54, 미기재 2, 미매핑 3(판교·성남시·김포), 중복 제거 1로 원천 60건이 남김없이 설명된다(7.4의 주간 롤업 실측).

미매핑 3건의 처리를 확인해야 한다. 판교, 성남시, 김포는 오타가 아니라 서울 자치구 사전에 없는 유효한 지명일 수 있다 — 자동으로 "가장 유사한 서울 자치구"에 매핑하는 것이 오히려 데이터 조작이다. 올바른 처리는 원문 그대로 분리 보고하고, 사람이 사전 보강(관할 확장)인지 반려(접수 오류)인지 판단하는 것이다. 실습의 일별 보고서는 이 3건을 "후속 조치 대상" 절에 민원 ID와 원문 표기로 싣는다.

핵심 코드

정규화와 매핑의 핵심 절차는 세 줄이다. 실패한 레코드를 버리지 않고 별도 리스트에 저장한다.

```

name = (raw or "").strip()           # 공백 보정
name = ALIASES.get(name, name)       # 약칭 보정: "강남" → "강남구"
if name in mapping: mapped.append(...) # 표준 코드 부여
elif not name:      missing.append(cid) # 미기재 - 분리 기록
else:               unmapped.append(...) # 매핑 실패 - 원문 보존

```

전체 코드는 *practice/chapter6/code/6-1-daily-report-dag.py* 참고

현장의 방법: 총계 보존식은 수작업으로 작성한 expectation이다

이 실습은 총계 보존을 파이썬 한 줄(`physical == mapped + missing + unmapped + dup_removed`)로 직접 검사한다. 규모가 커지고 규칙이 늘어나는 현장에서는 이런 검사를 매번 수작업으로 작성하는 대신, **데이터 품질 검증을 선언적으로 관리하는 프레임워크**를 사용한다. 두 계열을 파악하면 이 실습의 검사가 업계 표준의 축소판임이 확인된다.

첫째, **Great Expectations** 같은 데이터 검증 프레임워크는 “이 컬럼은 null이 없어야 한다”, “이 값은 정해진 집합 안에 있어야 한다” 같은 기대(expectation)를 코드가 아니라 규칙 목록으로 선언하고, 데이터가 통과했는지를 리포트로 남긴다. 이 실습의 “표준 코드 사전에 있는 값만 매핑에 들어간다”는 규칙은 GX 식으로 표현하면 “region_std는 법정동코드 사전의 값 집합에 속해야 한다”는 expectation 하나다. 둘째, **dbt의 데이터 테스트**는 변환 파이프라인의 결과 표에 대해 유일성·비결측·참조 무결성 같은 검증을 테스트로 선언하고, 실행하면 각 테스트가 통과/실패와 실패 행 수를 반환한다. 이 실패 상태를 CI나 오케스트레이터의 게이트로 연결하면 검증에 실패한 데이터가 하류로 흐르지 못하게 막을 수 있다 — 7.3의 품질 게이트가 배치를 실패시켜 틀린 보고서 생성을 막는 것과 정확히 같은 발상이다. 두 도구 모두 이 책의 실습과 대상·규모는 다르므로 직접 선례가 아니라 **방법론 차용**으로 참고한다. 요점은 도구가 아니라 태도다: 정제의 성공을 “에러가 안 났다”가 아니라 “선언한 기대를 데이터가 만족했다”로 판정하고, 그 판정을 기록으로 남긴다는 것. 이 실습의 총계 보존식·품질 게이트는 그 태도를 프레임워크 없이 수작업으로 구현한 최소판이다.

같은 프레임워크라도 민간이 사용할 때와 공공이 사용할 때 목적의 무게가 다르다. 민간에서 품질 게이트의 일차 목적은 틀린 데이터가 하류로 흐르지 못하게 파이프라인을 중단하는 것이다. 공공에서는 거기에 하나가 더 추가된다 — 몇 건이 왜 누락되었는지를 보고할 수 있어야 한다. 중단하는 것으로 끝나지 않고 누락의 내역이 산출물로 남아야 한다는 요구가, 이 실습이 미매핑을 원문 그대로 분리 기록하는 이유다.

공공 사례

두 기관이 “구별 민원 통계”를 교환하는데 한쪽은 “강남구”, 다른 쪽은 “서울 강남”으로 적는다면, 이름 기반 결합은 매번 수작업 보정을 요구하고 보정 규칙은 담당자마다 달라진다. 법정동코드로 교환하면 결합은 기계적이 된다. 검수 관점의 질문은 두 가지다: 납품 시스템이 지역·기관·품목 식별에 표준 코드를 쓰는가(아니라면 어떤 근거로 자체 코드를 쓰는가), 그리고 코드 매핑 실패분은 어디에 어떻게 보고되는가. 두 번째 질문에 “실패는 없도록 처리했다”는 답이 돌아오면 — 그것이 바로 조용한 drop의 신호다. ### 핵심 정리 - 정제는 표기 변형을 규칙(공백·약칭·코드 매핑)으로

흡수하고, 모든 레코드의 행방을 기록한다. - 매핑 실패는 버리지도, 자동으로 끼워 맞추지도 않는다 — 원문 보존·분리 보고가 원칙이다. - 총계 보존식은 “조용히 유실된 레코드”를 검출하는 한 줄짜리 검수 장치다.

7.3 Airflow DAG의 태스크와 의존성

학습 목표

배치 절차를 태스크의 방향 그래프(DAG)로 선언하는 방식을 이해한다.
logical date·실패 전파·재실행을 실측으로 확인한다.

직관: 절차를 코드가 아니라 그래프로 선언한다

7.2의 정제와 집계를 하나의 파이썬 스크립트로 이어 붙일 수도 있다. 그러나 운영에서는 곧 다음 질문이 제기된다 — 어제 새벽 배치가 집계 단계에서 중단되었는데 어디서부터 다시 실행하는가, 정제가 실패했는데 보고서 단계가 낡은 파일로 실행되지 않았는가, 매일 새벽 2시에 누가 실행하는가. 워크플로 오케스트레이터(orchestrator)인 Airflow는 이 질문들을 구조로 답한다. 절차를 **태스크**(무엇을)와 **의존성**(어떤 순서로)의 그래프로 선언하면, 엔진이 순서 보장·실패 전파·재시도·스케줄을 맡는다. 6장에서 “질의만 선언하면 증분 계산은 엔진이 맡는다”고 했던 것과 같은 구도다 — 선언과 실행의 분리.

그래프가 **방향 비순환 그래프(DAG, Directed Acyclic Graph)** 여야 하는 이유도 직관적이다. 방향은 “정제가 끝나야 집계한다”는 순서이고, 비순환은 “A가 B를 기다리고 B가 A를 기다리는” 교착이 없다는 보장이다. 실습 7.1의 DAG는 다음과 같다 — 이것이 이 장의 배치 워크플로 다이어그램이다.

```
graph LR
    validate_input --> clean_and_map
    clean_and_map --> aggregate_daily
    clean_and_map --> write_report
    aggregate_daily --> check_quality
    check_quality --> write_report
```

daily_complaint_report (@daily): 검증 → 정제·매핑 → (집계 || 품질 게이트) → 보고서
weekly_complaint_rollup (@weekly): rollup_weekly (일별 산출물 합산)

가운데의 분기 구조를 확인한다. 집계(aggregate_daily)와 품질 점검(check_quality)은 둘 다 정제 결과만 있으면 되고 서로를 필요로 하지 않는다 — 그래서 의존성을 선언하지 않았다. 보고서(write_report)는 둘 다 필요하므로 두 분기가 다시 합쳐진다. 의존성 선언은 코드 한 줄이다.

```
t_validate >> t_clean >> [t_aggregate, t_quality] >> t_report
```

전체 코드는 [practice/chapter6/code/6-1-daily-report-dag.py](#) 참고

태스크를 몇 개로 쪼갤 것인가

실습 DAG의 태스크는 5개지만, 같은 일을 1개로도, 10개로도 구성할 수 있다. 무엇이 이 경계를 정하는가. 판단 근거는 두 가지다 — **재실행 단위와 실패 격리**.

먼저 재실행 단위를 확인한다. 태스크는 실패했을 때 재실행하는 최소 단위다. 정제와 집계를 한 태스크로 묶으면 집계 태스크가 실패해도 정제 태스크부터 다시 실행한다. 정제에 10분이 걸리면 실패할 때마다 같은 시간이 추가로 소요된다. 반대로 한 줄짜리 연산까지 나누면 태스크 기동 부담(Airflow는 태스크마다 프로세스와 상태 기록이 붙는다)이 실제 계산보다 커진다. 그래서 경계는 “여기서 실패하면 앞 단계를 다시 실행하지 않아야 하는” 지점에 놓는다.

실패 격리는 다른 기준이다. 서로 다른 이유로 실패하는 일은 다른 태스크로 나눈다. 입력 검증은 파일이 없어서 실패하고, 품질 점검은 데이터가 규칙을 어겨서 실패한다. 둘을 한 태스크에 묶으면 실패 상태만으로는 원인을 구분할 수 없어, 7.4 체크리스트의 2단계(원인 분류)가 로그 전수 검색이 된다. 실습이 `check_quality` 를 집계에서 분리한 이유가 이것이다 — 품질 위반은 집계 실패와 다른 사건이고, 다른 대응을 부른다.

반대로 **묶어야 하는** 경우도 있다. 중간 산출물을 파일로 남길 이유가 없고 항상 함께 실패하는 단계들은 나눌 이유가 없다. 실습의 `clean_and_map` 이 공백 보정·약칭 보정·코드 매핑 세 규칙을 한 태스크에 담은 이유다 — 셋은 같은 입력을 순차 처리하는 하나의 변환이고, 중간 상태만 따로 재실행할 일이 없다.

logical date: “언제 돌아왔나”가 아니라 “어느 날짜의 데이터인가”

Airflow의 DAG 실행(run)에는 이름표가 붙는다 — **logical date**, 즉 이 실행이 처리하는 데이터 구간의 날짜다. 실습에서 7월 1일 데이터를 처리한 실행의 logical date는 2026-07-01이지만, 실제로 실행된 날짜는 7월 7일이다. 이 분리는 2장의 구분과 대응한다: 이벤트 시간(발생)과 처리 시간(도착)의 구분이 배치 세계로 오면 “데이터의 날짜”와 “실행된 시각”의 구분이 된다. 6장의 창 집계는 이벤트 시간 기준이어야 했던 것과 같은 이유로, 배치의 입력·산출물·재실행은 전부 logical date 기준이어야 한다.

이 이름표가 배치 운영의 핵심 능력 두 가지를 만든다. 첫째, **재실행**: “7월 1일 실행을 재실행하라”가 명확한 명령이 된다 — 어느 입력을 읽고 어느 산출물을 다시 쓸지 이름표가 정한다. 둘째, **소급 실행 (backfill)**: 파이프라인을 오늘 만들었어도 지난주 날짜들의 run을 소급 생성해 과거 구간을 채울 수 있다(보충). 실습이 7월 1~3일 사흘분을 오늘 실행한 것이 사실상 소규모 backfill이다.

backfill이 왜 실무의 필수 능력인지는 공공 시나리오로 옮기면 분명해진다. 새로 만든 일별 통계 파이프라인을 이번 달부터 가동한다고 가정한다. 그런데 부서장이 “지난 분기 것도 같은 기준으로 다시 뽑아 달라”고 요청한다 — 이전에는 담당자마다 다른 스크립트로 산출해 기준이 일관되지 않았기 때문이다. logical date가 없으면 이 요청은 “지난 90일분을 수작업으로 하나씩 실행하기”가 되지만, logical date 단위 backfill이 있으면 “2026-04-01부터 06-30까지 run을 생성하라”는 한 번의 명령이 된다. 그리고 이 소급 실행이 신뢰할 만하려면 — 즉 오늘 만든 코드로 과거 90일을 다시 계산했을 때 각 날짜가 “그날의 확정치”가 되려면 — 이 장 뒷부분의 재현성(같은 입력·같은 코드 → 같은 산출물, 7.4)이 전제되어야 한다. 소급 실행과 재현성은 한 쌍이다.

재실행과 소급 실행은 자주 같은 말로 쓰이지만 하는 일이 다르다. **재실행(rerun)**은 이미 생성된 run을 같은 logical date로 재실행한다 — 대상 날짜에 이미 산출물이 있거나 실패 상태가 남아 있으므로, 수정하는 작업이다. **소급 실행(backfill)**은 아직 생성된 적 없는 과거 날짜의 run을 새로 생성한다 — 없던 것을 생성하는 작업이다. 실무에서는 세 가지 차이가 나타난다. 첫째, 재실행은 덮어쓰기가 일어나므로 기존 산출물이 바뀐다(그래서 재현성이 없으면 어느 쪽이 진본인지 분쟁이 된다). 둘째, backfill은 한 번에 수십·수백 개 run을 만들 수 있어 동시 실행 수를 제한하지 않으면 원천 시스템에 부하가 몰린다. 셋째, 코드 이력이 다르다 — 재실행은 대개 같은 코드로 수행하지만, backfill은 오늘 코드로 과거 날짜를 계산하므로 그때 규칙과 지금 규칙이 다르면 과거 날짜의 값이 원래 보고한 값과 달라진다. 그래서 이미 배포한 확정 통계를 backfill로 다시 만들 때는 개정판임을 표기한다(7.1).

분리가 실제 기록에 어떻게 남는지 실습 증거로 확인한다(`ch6_batch_run_report.json`).

DAG run의 logical date	태스크가 실제로 실행된 날짜 (실측)	읽은 입력
2026-07-01	2026-07-07	<code>2026-07-01.jsonl</code>
2026-07-02	2026-07-07	<code>2026-07-02.jsonl</code>
2026-07-03	2026-07-07	<code>2026-07-03.jsonl</code>

사흘분 run 모두 실제 실행일은 같은 날(2026-07-07)이지만, 각 run은 자기 logical date의 입력만 읽고 자기 날짜의 산출물만 기록했다. “언제 실행되었는가”는 세 run이 같고 “어느 데이터인가”는 세 run이 다르다 — 두 시간축이 독립이라는 것이 이 표 한 장에 들어 있다.

logical date를 더 정확히 정의하려면 **데이터 구간(data interval)** 개념이 필요하다. Airflow 공식 문서는 스케줄 실행에서 logical date를 그 run이 담당하는 데이터 구간의 **시작**을 가리키는 이름표로 설명한다 — 일 배치라면 “그날 00시부터 다음 날 00시 직전까지”라는 반열린 구간 [시작, 끝) 이 한 run의 처리 대상이고, 스케줄러는 이 구간이 닫힌 뒤에(즉 그날이 다 지난 뒤에) run을 생성한다. “7월 1일치는 7월 2일 새벽에 계산한다”는 달력 마감의 직관이 바로 이 구조다. 반면 수동 실행(트리거)에서는 logical date와 data interval의 관계가 스케줄 실행과 다를 수 있으므로, 공식 문서도 둘이 같다고 가정하지 말라고 안내한다. 실제로 이 실습의 Airflow 3.3에서 `dag.test()` 로 특정 날짜를 실행하면 구간의 시작·끝이 logical date 하루로 관찰됐지만, 실습 코드는 이 관찰에 의존하지 않고 두 경우를 방어적으로 분기한다. `rollup_weekly` 는 data interval의 끝이 logical date와 다르면 그 끝(배타적)의 전날을, 같으면 logical date 당일을 창 의 끝으로 삼아 7일 창을 계산한다 — 같은 코드가 스케줄·수동 어느 실행에서도 같은 “주”를 가리키게 하려는 설계다. logical date와 data interval을 혼동하면 이 창 계산이 하루씩 어긋난다는 것이 요점이다.

실측 관찰 1: 의존성은 지켜지고, 병렬 태스크의 순서는 보장되지 않는다

실습의 사흘치 성공 실행에서 태스크 시작 시각 순서를 보면(`ch6_batch_run_report.json`), 세 실행 모두 `validate_input` → `clean_and_map` → (분기) → `write_report` 의 의존성을 지켰다. 분기된 두 태스크(집계·품질 점검) 사이의 선후는 증거 파일의 start 시각으로 확인할 수 있다. **집필 과정에서 같은 코드를 반복 실행하자 이 선후가 실행마다, 날짜마다 뒤바뀌었다** — 어떤 실행에서는 집계가,

다른 실행에서는 품질 점검이 먼저였다. 오류가 아니다. 의존성을 선언하지 않은 두 태스크 사이의 상호 순서는 엔진이 보장하지 않으며, 실행 환경에 따라 병렬로 실행될 수도 있다. “선언한 순서만 보장된다”는 뜻이다. 분기된 태스크의 실행 순서에 암묵적으로 의존하는 코드(예: 품질 점검이 집계 산출물을 읽는 것)는 예고 없이 실패할 수 있다.

실측 관찰 2: 실패는 전파되고, 기록된다

입력 파일이 없는 날짜(7월 4일)의 실행을 의도적으로 수행했다. 결과는 다음과 같다 — 실제 실행의 태스크 상태다.

표 7.3 실패 전파 관찰(2026-07-04 실행, 실측)

태스크	상태	의미
validate_input	failed	입력 파일 없음 — 예외 발생으로 실패
clean_and_map	upstream_failed	상류 실패로 실행되지 않음
aggregate_daily	upstream_failed	"
check_quality	upstream_failed	"
write_report	upstream_failed	"
(DAG run 전체)	failed	하나의 실패가 run 전체의 실패로 기록

두 가지를 확인해야 한다. 첫째, 하류 4개 태스크는 “실행됐다가 실패”한 것이 아니라 **실행 자체가 되지 않았다**(upstream_failed) — 입력이 없는데 보고서 태스크가 낡은 산출물로 실행되는 사고를 의존성 선언이 구조적으로 막았다. 둘째, 실패가 상태로 기록되었다 — 4장의 조용한 유실, 6장의 조용한 폐기와 대조적으로, 오케스트레이터의 실패는 명시적으로 기록된다. 어느 날짜의 어느 태스크가 왜 실패했는지가 남으므로, 운영자는 “입력 도착 후 7월 4일 run만 재실행”이라는 정확한 복구를 할 수 있다.

품질 게이트도 같은 구조를 사용한다. 실습의 `check_quality` 는 총계 보존식이 성립하지 않으면 예외를 발생시켜 실패한다 — 그러면 보고서 태스크가 upstream_failed로 중단되므로, **틀린 보고서는 생성 자체가 되지 않는다**. “일단 생성하고 나중에 수정하는” 것보다 “조건이 충족되지 않으면 중단하는” 것이 확정 산출물에서는 적절하다.

이것은 임의의 방법이 아니라 Airflow 공식 문서가 권장하는 패턴이다. Best Practices 문서의 “Self-Checks” 절은 “태스크가 기대한 결과를 냈는지 확인하는 검사를 DAG 안에 구현하라”고 명시하고, 예로 앞선 태스크가 S3에 데이터를 올렸다면 **다음 태스크에서 그 산출물(파티션 생성 여부 등)을 검사**하는 구조를 든다. 이 실습의 `check_quality` 가 하는 일이 정확히 그것이다 — 정제 태스크의 산출물(총계 보존식)을 별도 태스크가 검사하고, 어긋나면 파이프라인을 중단시킨다. 자기 검사를 파이프라인 안에 태스크로 포함하는 이 방식은 검사가 사람의 확인이 아니라 실행의 일부가 되게 하며, 그래서 담당자가 바뀌어도 누락되지 않는다.

게이트를 설정했으면 게이트가 얼마나 자주 발동하는지도 확인한다. 전체 실행 중 품질 게이트로 실패한 비율이 **게이트 발동률**이고, 이 값이 상승하는 추세는 원천 데이터 품질이 저하되고 있다는 신호다. 발동률이 0에 머무는 것도 정보다 — 데이터가 깨끗하다는 뜻일 수도 있고, 게이트가 실제로 동작하는지 한 번도 확인된 적이 없다는 뜻일 수도 있다. 규칙을 의도적으로 위반한 입력을 한 번 투입해 배치가 중단되는지 확인해 두면 둘을 구분할 수 있다. 알림이 발생하지 않는 것이 정상 상태인지 장애인지 구분해야 하는 6.4의 문제가 여기서도 같은 형태로 나타난다.

재시도 횟수와 간격은 무엇에서 정하는가

실패가 전파된다는 것은 확인했다. 그런데 모든 실패를 사람이 받아야 하는 것은 아니다. Airflow는 태스크마다 재시도 횟수(retries)와 간격(retry_delay)을 선언할 수 있고, 두 값은 **실패의 성질**에서 나온다.

재시도가 의미 있는 실패는 저절로 복구되는 실패뿐이다. 원천 DB 연결이 순간 끊긴 것, 네트워크 타임아웃, 일시적 잠금 충돌은 몇 분 뒤 재실행하면 성공한다. 반면 입력 파일이 없는 실패(실습의 7월 4일)는 재시도로 복구되지 않는다 — 파일이 도착하기 전에는 몇 번을 실행해도 결과가 같다. 그래서 횟수는 “이 태스크가 일시 장애로 실패할 여지가 있는가”로 정하고, 없으면 0이 맞다. 0으로 두면 실패가 곧바로 담당자에게 전달된다.

간격은 **장애가 복구되는 시간**에서 정한다. 원천 파일이 보통 몇 분 늦는다면 5분 간격 2~3회가 그 지연을 흡수하지만, 야간 적재가 30분 늦는 일이 있는 환경에서는 5분 간격 3회를 모두 소진해도 대기 시간이 부족하다. 총 대기 시간(횟수 × 간격)이 예상 복구 시간을 덮되 산출물 필요 시각을 넘기지 않아야 한다 — 7.1의 실행 시각 역산에서 설정한 복구 여유가 여기서 사용된다.

전제가 하나 있다. 재시도는 **역등한 태스크에만 안전하다**. 산출물을 절반만 기록하고 중단된 태스크가 재실행되며 앞서 기록한 내용에 이어 붙이면, 재시도가 이중 집계를 발생시킨다. 실습이 산출물을 덮어쓰기로 기록한 것(7.4)은 재현성의 조건이자 재시도의 안전 조건이다.

스케줄 선언과 실행 생성은 별개다

실습 DAG는 `schedule="@daily"` 와 `catchup=False` 를 선언한다. 그런데 실습에서 run을 생성한 것은 스케줄이 아니라 날짜를 지정한 `dag.test()` 호출이다 — 스케줄은 “운영 배포에서 스케줄러가 언제 run을 만들어야 하는가”의 선언이고, run의 생성 자체는 스케줄러·수동 트리거(trigger)·소급 실행 어느 쪽으로도 일어날 수 있다. `catchup=False` 는 그중 스케줄러의 행동에 대한 선언이다: 활성화 시점 이전의 미실행 구간을 자동으로 소급 생성하지 말라는 것. 과거 구간이 필요하면 backfill로 명시적으로 채우는 편이, 활성화하자마자 수백 개 run이 생성되는 사고보다 통제 가능하다(보충).

주기와 실행 시각을 정했으면 감시할 것이 하나 더 생긴다 — **언제 끝났는가**. 민간 데이터 조직은 마감 배치의 완료 시각(day-close completion time)을 운영 지표로 둔다. “오전 7시까지 전날 확정 매출을 경영진 대시보드에 올린다”는 약속이 있으면, DAG 마지막 태스크의 종료 시각이 그 약속의 측정값이다. 공공의 “매일 아침 8시까지 전날 민원 통계 배포”도 같은 구조이고 측정 방법도 같다. 완료 시각이 조금씩 지연되는 추세는 데이터가 증가하고 있다는 신호이므로, 약속 시각을 넘기기

전에 태스크를 나누거나 자원을 늘린다(11장). 오케스트레이터가 Airflow가 아니라 Dagster나 Prefect여도 이 지표와 감시 방식은 그대로다 — 도구가 아니라 배치 운영이 요구하는 것이기 때문이다.

실행 방식에 대한 각주

실습은 상주 스케줄러·웹서버 없이 `dag.test()` 로 DAG를 실행한다. 이것은 Airflow 공식 문서가 제공하는 디버깅·테스트 방식으로, DAG 전체를 단일 파이썬 프로세스에서 실행한다 — 태스크 인스턴스·상태·의존성 처리 모두 실제 엔진의 것이므로 위의 관찰은 전부 실제 동작이다. 다만 단일 프로세스이므로 분기된 두 태스크가 물리적으로 동시에 실행되지는 않는다(운영 배포에서는 executor가 병렬 실행). 상주 스케줄러를 생략해 수업 환경의 실행 부담을 줄였다. 스케줄러 기반 운영과 backfill은 보충 자료에서 설명한다.

공공 사례

“매일 아침 8시까지 전날 민원 통계를 각 부서에 배포한다”는 업무를 가정한다. 담당자가 수작업으로 스크립트 세 개를 순서대로 실행하는 구조에서는 순서 실수·누락·휴가가 전부 사고 요인이고, 무엇보다 실패가 기록되지 않는다. DAG로 선언하면 순서는 그래프가, 실행은 스케줄이, 실패는 상태 기록이 맡는다. 발주·검수 문서에서 “자동화되어 있다”는 문장을 확인하면 물어야 한다: 실패는 어떤 상태로 기록되며, 특정 날짜만 재실행하는 절차가 있는가. 이 두 질문에 답할 수 없는 자동화는 “cron에 스크립트를 걸어 둔 것”이지 운영 가능한 워크플로가 아니다. ### 핵심 정리 - DAG는 배치 절차의 선언이다: 태스크(무엇을) + 의존성(어떤 순서로), 실행·전파·기록은 엔진이 맡는다. - 실행의 이름표는 logical date(데이터의 날짜)이며, 재실행·소급 실행의 단위가 된다. - 선언한 의존성만 보장된다 — 실패는 하류를 `upstream_failed`로 멈추고, 상태로 기록된다.

7.4 일별·주별 리포트 자동화

학습 목표

보고서 파이프라인의 입력·출력을 계약으로 정의한다.
재실행 안전성(재현성)을 해시 비교 실측으로 검증한다.

입력과 출력은 계약이다

자동화 파이프라인이 장기간 유지되려면 입력과 출력이 암묵적 관행이 아니라 명시된 계약이어야 한다. 실습 7.1의 계약은 다음과 같다.

구분	계약	실습에서의 값
입력 경로	날짜별 원천 파일의 위치·이름 규약	<code>data/input/complaints/{YYYY-MM-DD}.jsonl</code>
입력 형식	필드·타입(1장의 스키마 관점)	<code>complaint_id, region_raw, category, created_at</code>
출력(일별)	확정 요약 + 품질 기록 + 보고서	<code>daily/{날짜}/daily_summary.json , quality_check.json , report.md</code>
출력(주별)	일별 산출물의 롤업(rollup, 창 끝 날짜로 파티셔닝(partitioning))	<code>weekly/{창 끝 날짜}/weekly_summary.json , weekly_report.md</code>
실패 규약	입력 없음·품질 위반 시 확정 마커(<code>_SUCCESS</code>)를 만들지 않음 — 부분 산출물은 남을 수 있으나 마커 없는 날짜는 후속 집계 대상이 아님	7.3의 실패 전파·품질 게이트

이 표가 곧 검수 자산이다. 입력 계약이 있으면 “수집 시스템이 파일을 어디에 어떤 이름으로 놓아야 하는가”가 두 팀 사이의 합의가 되고, 출력 계약이 있으면 후속 시스템(대시보드·기관 보고)이 무엇을 기대해도 되는지가 정해진다. 계약 없는 자동화는 담당자의 암묵적 규약에 의존하며, 인사이동이 있으면 함께 소실된다.

재현성: 같은 입력, 같은 보고서 — 바이트 단위로

배치가 재실행 가능하려면(7.1) 같은 날짜를 재실행했을 때 같은 결과가 나와야 한다. 실습은 이것을 가장 강한 형태로 검증했다: 7월 1일 실행의 산출물 해시를 기록하고, 같은 날짜를 통째로 재실행한 뒤 해시를 다시 계산했다. 실측 결과, `daily_summary.json` (sha256 앞 16자리 `88a34af5412b4723`) 과 `report.md` (`ba2787da6c5b8a47`) 모두 재실행 전후가 **완전히 동일**했다 — 5장에서 리플레이 후 업무 상태 지문이 같았던 것과 같은 종류의 증빙이다.

이 성질은 자동으로 확보되지 않는다. 실습 코드는 세 가지를 의도적으로 지켰다. 첫째, 보고서에 생성 시각 같은 비결정 요소를 넣지 않았다 — 내용은 입력의 순수 함수다. 둘째, JSON 직렬화의 키 순서를 고정했다 — 같은 내용이 다른 바이트가 되는 것을 막는다. 셋째, 산출물은 추가(append)가 아니라 덮어쓰기(overwrite)다 — 재실행이 이중 집계를 만들지 않는다(5장 Upsert의 파일판). 셋 중 하나라도 어기면 “재실행했더니 보고서가 달라졌다”가 되고, 그 순간 어느 쪽이 진본인지의 분쟁이 시작된다. 공공 보고서처럼 소급 감사가 있는 산출물에서 재현성은 품질이 아니라 방어력이다.

7.1에서 설명한 마감(day close)의 필요성은 이 절의 재현성 실측으로 검증된다. 민간의 정산 배치가 “어제의 확정 매출”을 만들고 공공의 결산·통계 마감에 “그 달의 확정치”를 만들 때, 확정치가 확정치인 이유의 절반은 “다시 계산해도 같다”는 데 있다. 재현성이 없는 확정치는 형용모순이다 — 감사관이 수치의 재현을 요구했을 때 다른 값이 나오면, 원래 보고한 값과 재현한 값 중 무엇이

진짜인지 아무도 답할 수 없고 마감 자체를 신뢰할 수 없게 된다. 그래서 이 해시 동일성 실측은 단순한 기술 확인이 아니라, 7.1이 말한 “늦어도 되지만 바뀌면 안 되는” 확정치의 조건을 코드로 증명한 것이다.

재현성은 한 번 확인하고 끝나는 성질이 아니다. 정제 규칙을 고치거나 보고서 서식을 바꾸면 그날부터 같은 입력이 다른 바이트를 낸다. 그래서 민간 데이터 조직은 이 해시 비교를 CI(Continuous Integration, 지속 통합) 파이프라인의 테스트로 포함한다 — 고정된 표본 입력의 산출물 해시를 기준값으로 저장해 두고, 코드가 바뀔 때마다 다시 계산해 비교한다. 값이 달라지면 빌드가 실패하므로, 의도한 변경인지 사고인지를 병합 전에 구분한다. 공공에서도 같은 장치를 사용할 수 있고, 여기서는 그 판정 기록이 “확정치의 계산 규칙이 바뀐 날짜”의 근거가 된다.

주별 롤업은 원천을 다시 읽지 않는다

주간 보고서를 만드는 두 가지 방법이 있다. 원천 7일분을 처음부터 다시 정제·집계하거나, 이미 확정된 일별 산출물 7개를 합산하거나. 실습은 후자를 택했고, 이것이 계층화의 원칙이다: **각 계층은 아래 계층의 확정 산출물만 읽는다.** 일별 산출물은 품질 게이트를 통과한 검증된 확정치이므로, 주간이 원천을 재정제하면 비용을 이중으로 소모할 뿐 아니라 정제 규칙의 버전 차이로 일별과 주간이 어긋날 위험까지 생긴다.

그렇다면 “확정”은 무엇으로 판별하는가. 파일이 존재한다는 것만으로는 부족하다 — 어떤 날짜의 재실행이 중간에 실패하면, 이전 성공이 남긴 낡은 파일이 최신 확정치처럼 보일 수 있기 때문이다. 실습은 성공 마커(`_SUCCESS`) 규약을 사용한다: 일별 DAG의 첫 태스크가 재실행 시작 시 그 날짜의 마커를 제거하고, 마지막 태스크까지 성공했을 때만 다시 기록한다. 주간 롤업은 마커가 있는 날짜만 합산하고, 없는 날짜는 결측으로 기록한다 — “산출물 세트의 완결”을 파일 존재가 아니라 명시적 증표로 판정하는, 배치 파이프라인의 오랜 관례다.

주간 롤업의 실측을 확인한다(`weekly_summary.json`). 사흘 합산 결과 원천 60건 = 매핑 54 + 미기재 2 + 미매핑 3 + 중복 제거 1로 총계 보존이 주간 수준에서도 성립했고, 지역별로는 강남구(11680) 23, 마포구(11440) 16, 관악구(11620) 15 — 일별 매핑 건수 $18+16+20=54$ 와 정확히 일치한다. 유형별로는 도로 21, 소음 15, 환경 10, 청소 8이다. 일별 보고서의 수치를 그대로 합산해 주간이 산출된다는 이 자명해 보이는 성질은, 사실 일별이 확정 산출물이고 주간이 그것만 읽기 때문에 성립하는 설계의 결과다.

이 계층화는 공공에만 필요한 규율이 아니다. 민간의 경영 보고도 일별 확정치를 주별·월별·분기별로 올려 쌓고, 상위 집계가 원천을 다시 읽으면 같은 문제가 생긴다 — 월간 보고의 합계가 일별 보고의 합과 맞지 않는데 양쪽 다 “제대로 계산된” 상황이다. 계층을 어기는 순간 두 보고서의 정합은 우연에 맡겨진다.

배치 실패 대응 체크리스트

배치는 실패한다 — 입력 지연, 원천 형식 변경, 품질 게이트 발동, 인프라 장애. 중요한 것은 실패했을 때의 절차가 미리 문서화되어 있는가다. 아래 체크리스트는 실습의 실측 관찰(실패 전파·품질 게이트·재현성)에 대응 절차를 붙인 것으로, 이 장의 정본 검수 자산이다.

표 7.4 배치 실패 대응 체크리스트

단계	점검 항목	실습에서의 근거
1. 발견	실패한 run의 날짜(logical date)와 태스크를 특정했는가	7/4 run: validate_input failed
2. 원인 분류	입력 문제(없음·지연·형식) / 품질 게이트 / 코드 결함 / 인프라 중 무엇인가	입력 파일 없음 (FileNotFoundException)
3. 영향 범위	하류 태스크·후속 보고가 낡은 산출물로 실행되지 않았는가	하류 4개 모두 upstream_failed — 산출물 미생성
4. 재실행 판단	원인 해소 후 해당 날짜만 재실행하면 되는가, 이후 날짜도 영향인가	logical date 단위 재실행(7.3)
5. 재실행 안전	재실행이 이중 집계·산출물 분기를 만들지 않는가	해시 동일 실측(재현성)
6. 상위 산출물	주간 등 롤업 계층의 재생성이 필요한가	주간은 일별 산출물만 읽음 — 일별 복구 후 재롤업
7. 재발 방지	같은 원인의 실패를 알림· 검증으로 앞당길 수 있는가	입력 검증 태스크가 첫 관문(11장 모니터링으로 확장)

공공 사례

월요일 아침, 지난주 주간 보고서의 수요일 수치가 이상하다는 지적이 왔다고 가정한다. 체크리스트가 있으면 대응은 기계적이다: 수요일 run의 품질 기록(quality_check.json)을 열어 미매핑·미기재가 급증했는지 확인하고, 원인이 원천이면 수집 쪽에 이관, 정제 규칙이면 규칙을 고쳐 수요일 하루만 재실행하고, 주간을 재롤업한다. 각 단계의 산출물이 남아 있으므로 “누가 언제 무엇을 재실행했는지”까지 설명 가능하다. 체크리스트가 없으면 같은 상황은 전체 재실행으로 바뀌고, 재현성이 없으면 재실행한 결과가 원본과 달라지는 두 번째 사고로 이어진다.

역방향: 이 장의 배치 설계를 민간 현장에 옮기면

7.1에서는 민간의 마감 배치를 공공의 확정 보고로 옮겼다. 반대 방향도 성립한다 — 이 장에서 만든 5태스크 DAG와 계약·재현성 규약을 그대로 민간 현장에 적용하면 무엇이 유지되고 무엇이 바뀌는지 세 현장에서 확인한다.

일매출 집계와 정산 파이프라인은 이 장의 일별 DAG와 골격이 같다. 자정이 지나면 그날의 결제·환불·취소를 전량 읽어 최종 매출을 확정하고, 확정 매출은 회계 원장에 기록된다. 마감 시각을 자정이 아니라 자정+N시간으로 설정하는 이유도 실행 시각 역산 그대로다 — 환불 이벤트가 결제보다 늦게 도착하기 때문이다. 소급 정정을 별도 정정 배치로 처리하고 원본을 덮지 않는 것도 공공의 개정판 절차와 같은 발상이다. 바뀌는 것은 근거다. 공공에서 확정치를 못 바꾸는 이유가 감사·정보공개라면, 여기서는 이미 마감한 회계 기간을 흔들 수 없다는 회계 규율이다.

추천 모델 재학습 배치는 태스크가 길어질 뿐 구조가 같다: `validate_logs` → `clean_and_map` → `build_features` → `train_model` → `evaluate` → `deploy`. 앞의 두 태스크는 이 장의 것과 이름까지 같고, `evaluate` 가 `check_quality` 자리에 들어간다 — 성능이 기준 이하면 `deploy` 를 `upstream_failed`로 멈춰, 나쁜 모델이 나가지 못하게 한다. 품질 게이트가 “틀린 보고서를 만들지 않는” 장치에서 “나쁜 모델을 내보내지 않는” 장치로 바뀔 뿐 원리는 하나다. logical date로 재학습 날짜를 관리하면 특정 날짜의 재학습을 그대로 다시 돌릴 수 있다는 점도 같다(9장).

야간 ETL 적재와 BI 리포트 자동화는 7.2와 7.4가 함께 옮겨 가는 현상이다. 여러 운영 DB(주문·재고·회원·물류)의 데이터를 새벽에 추출해 데이터 웨어하우스에 적재할 때, 상품명·카테고리의 표기 변형을 SKU로 통일하는 일이 7.2의 표준 코드 매핑에 해당한다. 적재된 일별 확정 데이터를 주별·월별로 롤업해 경영 보고서를 만드는 일은 7.4의 계층화다. 검증도 이름만 다르다 — 수작업으로 작성한 총계 보존식이 dbt test로 선언되고, 실패는 같은 방식으로 하류를 막는다.

공통 원칙이 있다. 태스크 분할 경계, 스케줄 주기와 실행 시각, 재시도 정책, 멱등성 확보 방식 — 이 네 결정은 공공이든 민간이든 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사·형평성·법적 보고 의무라면, 민간에서는 매출·비용·SLA(서비스 수준 약정)다. 방향은 여기서도 비대칭이다. 민간 파이프라인을 공공에 도입할 때는 재현 증거·원천 스냅샷·개정 이력이라는 층이 하나 추가되지만, 공공 설계를 민간에 도입할 때는 그 층을 제외해도 손실이 없다 — 재현성 검증은 CI 테스트가 되고, 미매핑 분리 보고는 마스터 데이터 보강 목록이 된다.

핵심 정리

입력·출력 계약의 문서화가 자동화를 담당자 의존에서 구조 의존으로 바꾼다.

재현성(같은 입력 → 바이트 동일 산출물)은 결정적 내용·고정 직렬화·덮어쓰기로 만들어지며, 해시로 검증한다.

롤업 계층은 확정된 하위 산출물만 읽고, 실패 대응은 체크리스트로 절차화한다.

실습 7.1 일별 민원 요약 DAG 작성(★★★☆☆)

이 실습은 **설계** → **실행** → **대조** → **위험 식별** 네 단계로 진행한다. 이미 정해진 DAG를 그대로 실행하면 태스크가 왜 5개인지, 재시도를 왜 그 값으로 두었는지 묻지 않게 된다. 먼저 자기 판단으로 분할 경계와 운영 정책을 정하고 실패 전파를 예측한 다음, 실행 결과와 대조한다. 어긋난 지점이 곧 자기 설계의 결함이다.

실습 목표

태스크 분할 경계·스케줄 주기·실행 시각·재시도 정책을 스스로 정하고 근거를 적는다(설계).

검증→정제·매핑→집계/품질→보고서의 5태스크 DAG와 주간 롤업 DAG를 작성하고 실행한다(실행).

입력이 없는 날짜의 실패 전파를 실행 전에 예측하고, 실측 상태와 대조한다(대조).

의존성 준수·실패 전파·재실행 재현성을 각각 실측 증거로 확인한다(증거).

입력 데이터

민원 이벤트 사흘분(물리 레코드 60건)은 시뮬레이션 입력이다(4·5·6장과 같은 이유 — 개인정보가 포함된 실제 민원을 사용할 수 없다). 표기 변형·결측·비표준 지역·중복 접수는 검산이 가능하도록 의도적으로 포함시켰다. 이 실습의 관찰 대상은 데이터 내용이 아니라 **실제 Airflow 엔진의 의존성·실패 전파·재실행 동작**이며, 인용 수치는 모두 실제 실행 산출물에서 산출된다. 표준 코드(법정동코드 시군구 5자리)는 행정표준코드관리시스템(code.go.kr)의 실제 코드값이다.

1단계 설계 — 태스크 분할과 운영 정책 결정

코드를 실행하기 전에 자기 판단으로 설계를 적는다. 실습 코드의 DAG는 완성된 예시 답안이므로 먼저 보고 옮겨 적지 않는다 — 채운 뒤에 비교한다.

요구사항(가상 발주 조건) - 민원 접수 원천은 야간 적재로 매일 새벽 1시경 날짜별 파일로 도착한다. 늦으면 새벽 2시를 넘기는 날도 있다. - 각 부서는 매일 아침 8시 회의에서 전날 확정 통계를 사용한다. - 지역명 표기가 일관되지 않으므로 표준 코드로 변환해야 하고, 사전에 없는 표기는 폐기하지 말고 따로 보고해야 한다. - 감사 요구: 특정 날짜의 통계를 나중에 다시 만들었을 때 같은 값이 나와야 한다. - 원천 파일이 전혀 도착하지 않는 날이 있다.

표 7.5 배치 운영 설계표 — 학습자 작성용

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
태스크 분할(몇 개로, 경계는 어디에)		
스케줄 주기		
실행 시각		
태스크별 재시도 횟수·간격		
역등성 확보 방식		
이 설계가 깨지는 조건		

작성할 때 지킬 것이 셋이다. 첫째, 근거 칸에 “기본값이라서” 또는 “보통 그렇게 한다”를 쓰지 않는다. 둘째, 실행 시각은 원천 확정 시각과 산출물 필요 시각 사이에서 역산한다(7.1). 셋째, 재시도는 태스크마다 다르게 정해도 된다 — 다섯 태스크에 같은 값을 일괄로 주었다면 왜 그래도 되는지를 근거 칸에 적는다.

실패 전파 예측: 실행 스크립트의 시나리오 C는 입력 파일이 없는 7월 4일을 실행한다. 실행하기 전에 다섯 태스크가 각각 어떤 상태로 끝날지 적어 둔다. 상태 이름을 모르면 “실행됨 / 실행 안 됨”으로 적어도 된다.

태스크	내 예측	실제(3단계에서 기입)
validate_input		
clean_and_map		
aggregate_daily		
check_quality		
write_report		

이어서 한 문장을 적는다 — “이 실행이 실패하면 재실행해야 하는 범위는 어디까지인가.” 태스크 하나인가, 그 날짜의 run 전체인가, 이후 날짜까지인가, 주간 롤업도 포함인가.

2단계 실행 — 선수 조건과 네 시나리오

Python 3.10 이상(이 실습은 3.13.7에서 검증)과 macOS/Linux가 필요하다 — Airflow는 Windows 네이티브 미지원이므로 Windows에서는 WSL2 또는 컨테이너로 실행한다. Airflow는 의존성이 많아 반드시 공식 constraints와 함께 설치하며(수 분 소요), constraints 파일 이름의 파이썬 버전은 **자기 인터프리터 버전에 맞춘다**(아래 예시는 검증에 사용한 3.13 기준 — 3.12를 사용한다면 constraints-3.12.txt).

```
cd practice/chapter6
python3.13 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt \
    --constraint https://raw.githubusercontent.com/apache/airflow/constraints-3.3.0/constraints-3.13.txt
python run_chapter6.py
```

실행기는 Airflow 환경(AIRFLOW_HOME)을 실습 폴더 내부로 고정하고(사용자 홈 오염 방지), 네 시나리오를 순서대로 실행한다.

시나리오	내용	관찰 목표
A	7/1~7/3 사흘분 일별 DAG 실행	의존성 순서·정제 통계(표 7.2)
B	7/1 재실행 후 산출물 해시 비교	재현성(7.4)
C	입력 없는 7/4 실행	실패 전파(표 7.3)
D	주간 롤업 DAG 실행	계층 합산·총계 보존

핵심 코드

DAG 선언과 테스트 실행의 기본 구조다. 스케줄은 선언하되(@daily), 실행은 dag.test() 로 특정 logical date를 지정해 수행한다.

```

with DAG(dag_id="daily_complaint_report", schedule="@daily",
         start_date=pendulum.datetime(2026, 7, 1, tz="UTC"), catchup=False) as dag:
    t_validate >> t_clean >> [t_aggregate, t_quality] >> t_report

dag.test(logical_date=pendulum.datetime(2026, 7, 1, tz="UTC"))

```

전체 코드는 [practice/chapter6/code/6-1-daily-report-dag.py](#) 참고

실행 중 확인 체크리스트

콘솔 출력에서 다음을 순서대로 확인하면 실습의 학습 목표를 모두 확인하게 된다.

- ☐ 시나리오 A: 사흘 모두 dagrun=success이고, 일별 매핑/미기재/미매핑/중복이 18·1·1·0, 16·0·1·1, 20·1·1·0인가
- ☐ 시나리오 B: 재실행 후 두 산출물의 해시가 동일한가(산출물 동일=True)
- ☐ 시나리오 C: validate_input만 failed이고 나머지 4개가 upstream_failed인가
- ☐ 시나리오 D: 주간 합산이 60 = 54+2+3+1로 보존되고 지역별 23·16·15인가
- ☐ 마지막 줄이 CH7_RUN_PASS 인가

산출물 안내

스크립트는 날짜별·주간·증거의 세 층위 산출물을 남긴다.

산출물	내용	용도
daily/{날짜}/report.md	사람이 읽는 일별 보고서(지역·유형 표, 매핑 실패 상세)	보고서 자동화의 최종 산출물
daily/{날짜}/daily_summary.json · quality_check.json	기계가 읽는 요약·품질 기록(같은 폴더의 _SUCCESS 가 있어야 확정)	주간 롤업·감사 추적의 입력
daily/{날짜}/excluded.json	미기재·미매핑·중복 제거의 상세(원문 보존)	후속 조치·분리 보고
daily/{날짜}/_SUCCESS	마지막 태스크 성공 시에만 기록되는 완결 마커(재실행 시작 시 제거)	주간 롤업의 합산 기준(7.4)
weekly/{창 끝 날짜}/weekly_report.md · weekly_summary.json	주간 롤업 보고서·요약 (주 단위 파티셔닝)	계층화 실측
ch6_batch_run_report.json	전 시나리오의 태스크 상태·해시·판정	이 장 인용 수치의 증거 파일

2단계 실행 결과(실제)

사흘분 일별 실행의 정제·집계 결과는 다음과 같다(시나리오 A, `ch6_batch_run_report.json`).

날짜	물리	중복제거	매핑	미기재	미매핑 (원문)	지역별 (강남·마포·관악)	최다 유형
7/1	20	0	18	1	1(판교)	8·5·5	도로(7)
7/2	18	1	16	0	1(성남시)	6·5·5	도로(7)
7/3	22	0	20	1	1(김포)	9·6·5	도로(7)

시나리오 B(재실행)는 `daily_summary.json` 과 `report.md` 의 sha256이 재실행 전후 완전히 동일했고(각 `88a34af5412b4723...`, `ba2787da6c5b8a47...`), 시나리오 C(7/4)는 표 7.3대로 `validate_input failed` + 하루 4개 `upstream_failed`로 run 전체가 `failed`였다. 시나리오 D(주간)는 $60 = 54 + 2 + 3 + 1$ 의 총계 보존과 강남 23·마포 16·관악 15, 유형별 도로 21·소음 15·환경 10·청소 8을 산출했다.

3단계 대조 — 예측과 실측의 대조

1단계 예측표의 “실제” 열을 시나리오 C의 결과(표 7.3)로 채운다. `validate_input` 은 `failed`, 나머지 넷은 `upstream_failed`다.

예측을 “다섯 개 모두 `failed`”로 적었다면 하나를 놓쳤다. 하루 넷은 실행됐다가 실패한 것이 아니라 **실행 자체가 되지 않았다**. 두 상태는 남는 산출물이 다르다 — 실행됐다면 일부만 작성된 보고서 파일이 남았을 수 있고, 그 파일이 다음 주간 롤업에 포함된다. 실행되지 않았으므로 그런 파일이 없고, `_SUCCESS` 마커도 없다.

예측을 “`validate_input`만 `failed`이고 나머지는 건너뛴”으로 적었다면 절반만 정확하다. 남은 절반은 그 건너뛴이 **상태로 기록된다**는 것이다. 상태가 없으면 “실행되지 않았다”와 “실행할 필요가 없었다”를 구분할 수 없고, 표 7.4의 1단계(실패한 날짜와 태스크 특정)가 성립하지 않는다.

재실행 범위에 대해 적은 문장도 지금 확인한다. 이 경우 답은 “그 날짜의 run 전체”다 — 입력이 없어 첫 태스크가 실패했으므로 파일이 도착한 뒤 7월 4일 run을 재실행한다. 다른 날짜는 각자 자기 logical date의 입력만 읽었으므로 영향이 없다. 다만 7월 4일이 주간 창 안에 들어가는 날짜라면 롤업은 다시 실행해야 한다. 그 날짜에 `_SUCCESS` 가 생기면 합산 대상이 하나 늘어 주간 수치가 바뀌기 때문이다.

마지막으로 설계표로 돌아간다. 시나리오 C가 확인한 것은 **입력 없음이 재시도로 복구되지 않는 실패**라는 사실이다. `validate_input` 의 재시도를 넉넉히 설정했다면, 그 재시도는 파일 도착을 기다리는 장치가 될 수도 있고(원천이 몇 분 늦는 경우) 실패 통보를 늦추기만 하는 장치가 될 수도 있다(파일이 전혀 도착하지 않는 경우). 어느 쪽인지는 원천의 지연 패턴이 정하며, 요구사항의 “늦으면 새벽 2시를 넘기는 날도 있다”가 그 근거다. 표 7.5의 재시도 칸과 “이 설계가 깨지는 조건” 칸을 이 관찰로 다시 쓴다. **제출하는 설계표는 이 수정을 반영한 최종본이다.**

관찰

7/2의 중복 접수(같은 민원 ID C260702-001의 재전송)는 정제 1단계에서 제거되어 물리 18건이 유효 17건이 되었다 — 5장의 중복 문제가 배치 정제의 첫 규칙으로 재등장한 것이다. 제거된 ID는 `excluded.json`에 남는다(폐기하지 않고 기록).

병렬인 두 태스크(집계·품질)의 상호 시작 순서가 고정되지 않았다 — 증거 파일의 태스크별 start 시각으로 각 run의 선후를 확인할 수 있는데, 집필 중 반복 실행에서 이 선후는 실행 날짜마다 달랐다(직접 실행한 뒤 자신의 증거 파일과 비교한다). 선언하지 않은 순서는 보장되지 않는다는 7.3의 명제가 그대로 관찰됐다.

미매핑 3건(판교·성남시·김포)은 세 날짜의 일별 보고서 “후속 조치 대상” 절에 원문 그대로 실렸다. 자동 보정 대신 분리 보고를 택한 결과, 사람이 판단할 근거(관할 확장인가, 접수 오류인가)가 산출물에 남았다.

실행 시간의 대부분은 Airflow 초기화(메타데이터 DB 마이그레이션·DAG 직렬화, 수 초)다. 태스크 5개의 실제 처리는 날짜당 1초 미만이며, 이 실습의 목적은 처리 성능이 아니라 오케스트레이션 의미론의 관찰이다.

증거 파일의 태스크별 시작 시각(start)이 곧 의존성 준수의 증거다 — 세 성공 run 모두 `validate` → `clean`이 앞서고 `report`가 마지막이다.

실행 후 `data/output/airflow_home/`에 메타데이터 DB가 남는다. 어느 run의 어느 태스크가 어떤 상태인지의 기록이며, 표 7.3의 상태들도 여기서 조회한 것이다. 4장의 커밋(commit)된 오프셋(offset), 6장의 체크포인트(checkpoint)에 이어 세 번째로 확인하는 “진행 위치의 기록”이다 — 도구가 바뀌어도, 재실행과 복구는 언제나 기록된 진행 상태를 전제한다.

실행 환경에서 발생한 오류 사례

이 실습을 준비하며 실제로 발생한 오류를 기록해 둔다. DAG 파일을 작성하고 곧바로 `dag.test()`를 호출하자 `Cannot create DagRun ... because the dag is not serialized` 오류로 실패했다. Airflow 3의 아키텍처에서 DAG 실행은 메타데이터 DB에 직렬화된 DAG 정의를 전제하기 때문으로, 해법은 실행 전에 DB 마이그레이션과 DAG 직렬화를 한 번 수행하는 것이다(실습 코드의 `bootstrap()` — `airflow db migrate + airflow dags reserialize`). “코드 파일이 있다”와 “엔진이 그 DAG를 안다”가 별개라는 것, 즉 오케스트레이터는 코드가 아니라 등록된 정의를 실행한다는 Airflow 3의 구조를 실제로 확인하는 오류였다.

4단계 위험 식별 — 설계와 증거를 검수 언어로 옮기기

앞의 세 단계에서 설계 근거(1단계)와 실행 증거(2·3단계)를 모두 확보했다. 검수·감사가 요구하는 것은 그 둘을 이어 붙인 형태다 — 무엇이 잘못될 수 있는가, 그렇게 판단한 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가. 앞의 네 행은 실행 증거에서, 마지막 행은 1단계 설계에서 나온다.

위험	실습에서의 근거	대응 방향
정제 단계의 조용한 레코드 유실	총계 보존식 검증($60 = 54+2+3+1$)	품질 게이트로 보존 위반 시 배치 실패
낡은 산출물로 하류 실행	7/4: 하류 전부 upstream_failed로 미실행	의존성 선언 + 실패 전파 확인
재실행 시 산출물 분기	해시 동일 실측	결정적 산출물 + 덮어쓰기 설계
미매핑의 자동 끼워 맞춤	판교·성남시·김포 분리 보고	원문 보존·후속 조치 절차화
원천 지연을 재시도로 흡수하려다 배포 시각을 넘김	1단계 설계: 실행 시각 역산과 재시도 총 대기(횟수 × 간격)	총 대기를 복구 여유 안에 두고, 초과분은 재시도가 아니라 실패 통보로 전환

대응 방향 칸이 “모니터링한다”로 채워지면 그 행은 검수 문서에서 유용하지 않다. **바꿀 설정값이나 바꿀 설계 결정**이 들어가야 한다.

제출물 3종: ① 표 7.5 배치 운영 설계표(3단계 수정을 반영한 최종본), ②

`ch6_batch_run_report.json`, ③ 위 위험 식별 표. ①이 설계, ②가 증거, ③이 둘을 이은 검수 자산이다.

보충(전문가 트랙 포인터)

backfill과 catchup 정책: 실습은 사흘분을 수동으로 소급 실행했지만, 운영 Airflow는 backfill 명령으로 과거 구간의 run들을 생성·실행하고, catchup 설정으로 시작일 이후 미실행 구간의 자동 따라잡기 여부를 정한다. 소급 실행이 안전하려면 이 장의 재현성(결정적 산출물·덮어쓰기)이 전제다.

SLA 알림과 실패 재시도 설계: 태스크에는 재시도 횟수·간격을 선언할 수 있다(일시 장애 흡수). 단 재시도는 멍든 태스크에만 안전하며, “8시까지 보고서가 나와야 한다”류의 기한은 지연 감지·알림으로 감시한다 — 알림 설계의 주의점은 6.4와 같다.

공공 보고서 산출물의 재현성 관리: 이 장은 같은 입력·같은 코드의 재현성을 다뤘다.

운영에서는 코드 버전(태그)·정제 규칙 버전·입력 스냅샷까지 함께 기록해야 “그때 그 보고서”를 재생성할 수 있다 — 9장(실험 추적·레지스트리)의 버전 관리 사고방식이 보고서에도 적용된다.

핵심 용어

배치 처리: 데이터 구간 마감 후 전체를 계산하는 처리 — 확정 통계·보고서의 기반

데이터 정제: 표기 변형·결측·중복을 규칙으로 흡수하고 모든 레코드의 행방을 기록하는 단계
표준 코드 매핑: 자연어 명칭을 표준 코드(법정동코드 등)로 바꿔 집계·결합의 키로 사용하는 것

총계 보존: 입력 = 매핑 + 미기재 + 미매핑 + 중복제거 — 조용한 유실을 검출하는 검증식
DAG: 태스크(무엇을)와 의존성(어떤 순서로)의 방향 비순환 그래프 — 배치 절차의 선언
logical date: 실행이 처리하는 데이터 구간의 날짜 — 재실행·소급 실행의 단위
upstream_failed: 상류 실패로 실행되지 않은 태스크 상태 — 실패 전파의 기록
품질 게이트: 검증 실패 시 배치를 실패시켜 틀린 산출물 생성을 막는 태스크
재현성: 같은 입력의 재실행이 바이트 동일 산출물을 산출하는 성질 — 해시로 검증
롤업: 하위 계층의 확정 산출물만 읽어 상위 집계를 산출하는 것(주간 = 일별의 합산)
backfill / catchup: 과거 구간 run의 명시적 소급 실행 / 스케줄러의 미실행 구간 자동 생성
정책
입력·출력 계약: 파이프라인이 읽고 쓰는 경로·형식·실패 규약의 명시적 합의 — 검수 자산

다음 장 예고

이 장까지로 데이터는 수집되고(4장), 중복이 제거되고(5장), 실시간으로 집계되고(6장), 배치로 정제·확정되었다(7장). 다음 단계는 이 데이터를 모델이 입력으로 받는 형태 — 피처(feature)로 바꾸는 것이다. 그런데 피처는 훈련할 때와 예측할 때 같은 값이어야 한다는 까다로운 요구가 있다. 훈련은 이 장의 배치 산출물처럼 과거 데이터에서, 예측은 6장의 스트리밍처럼 실시간으로 일어나기 때문이다. 8장은 이 훈련·서빙 일관성 문제를 푸는 피처 스토어를 다룬다 — 이 장의 일별 집계가 그대로 오프라인 피처의 재료가 된다.

참고문헌

Apache Airflow. (n.d.). *Dags — Core Concepts*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>
Apache Airflow. (n.d.). *Tasks — Core Concepts*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/tasks.html>
Apache Airflow. (n.d.). *Dag Runs — Core Concepts*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dag-run.html>
Apache Airflow. (n.d.). *Debugging Airflow Dags*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/debug.html>
Apache Airflow. (n.d.). *Backfill — Core Concepts*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/backfill.html>
Apache Airflow. (n.d.). *Best Practices*. <https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html> (7.3 품질 게이트의 직접 선례 — “Self-Checks” 절: 다음 태스크에서 산출물을 검사하는 패턴)
dbt Labs. (n.d.). *Add data tests to your DAG*. <https://docs.getdbt.com/docs/build/data-tests> (방법론 차용 — 변환 파이프라인에 테스트 게이트를 붙이는 도구, 본 실습과 도구·규모는 다름)
Great Expectations. (n.d.). *GX Overview*. https://docs.greatexpectations.io/docs/core/introduction/gx_overview/ (배경·방법론 차용 — expectation 기반 데이터 검증 프레임워크, 총계 보존식의 프레임워크판)

행정안전부. (n.d.). 행정표준코드관리시스템 — 법정동코드.

<https://www.code.go.kr/stdcode/regCodeL.do>

국토교통부. (2025). 행정구역법정동코드 (파일데이터). 공공데이터포털.

<https://www.data.go.kr/data/15122602/fileData.do>